Install and use ragcap by hand

Written for: setting up ragcap step by step on an Apple Silicon Mac, without the one-command script, and then using it day to day. Every step says what to download, where from, what to type, and what you should see. No admin password, Homebrew or Docker is needed. Date: 2026-10-23; every step was run on an Apple M5 Pro (macOS 26.6).

`./recreate.sh` does sections 3 to 5 in one command. Do it by hand when you want to understand each part or when the script stops with an error.

1. What you need, and where it comes from

Software	Why	Download from	Size	Admin rights?
uv	installs Python 3.12 into your home folder	https://astral.sh/uv (install script)	~40 MB	no
Python 3.12	runs the code	installed by uv	~60 MB	no
Ollama	runs the language models locally	https://github.com/ollama/ollama/releases (file `ollama-darwin.tgz`), or the app from https://ollama.com/download	~140 MB	no (standalone file); the app goes in Applications
llama3.2:3b, nomic-embed-text	the answering model and the search (embedding) model, required	downloaded by Ollama from https://ollama.com/library	2.3 GB	no
qwen3:8b	second-opinion grader, recommended	Ollama	5.2 GB	no
qwen2.5vl:3b	reads P&ID drawings, only if you load drawings	Ollama	3.2 GB	no
Postgres 16 + pgvector	the database	nothing to download by hand: the Python package `pgserver` (from https://pypi.org) brings it in step 3.4	~30 MB	no
Docker Desktop	optional alternative for the database	https://www.docker.com/products/docker-desktop/	~2 GB	yes; skip it

Disk needed: about 3 GB for the lean setup, 10.5 GB with every recommended model. Memory: about 3 GB while answering (8 GB for a few minutes when the second-opinion grader is used). Which models to keep and why: `docs/APPLE_SILICON.md` section 7.

2. Install the tools (once per Mac)

Open Terminal (Applications > Utilities > Terminal) and run each block. Lines starting with `#` are comments.

2.1 uv and Python 3.12

```
curl -LsSf https://astral.sh/uv/install.sh | sh        # installs uv into ~/.local/bin
source ~/.zshrc                                         # or open a new Terminal window, so ~/.local/bin is on PATH
uv python install 3.12                                  # Python 3.12 for Apple Silicon, into your home folder
python3.12 --version                                    # expect: Python 3.12.x
```

If `python3.12` is "command not found", use the full path `~/.local/bin/python3.12` in the steps below.

2.2 Ollama

Option A, no admin rights (what this Mac uses): the standalone program, kept in your home folder.

```
mkdir -p ~/.local/opt/ollama && cd ~/.local/opt/ollama
curl -fsSL -o ollama-darwin.tgz https://github.com/ollama/ollama/releases/latest/download/ollama-darwin.tgz
```

```
tar xzf ollama-darwin.tgz && rm ollama-darwin.tgz
ln -sf ~/.local/opt/ollama/ollama ~/.local/bin/ollama  # so the command `ollama` works anywhere
mkdir -p ~/.ollama && nohup ollama serve > ~/.ollama/serve.log 2>&1 &    # start the server in the background
ollama --version                                       # expect: ollama version is 0.40.x (or newer)
```

Option B, the app: download `Ollama.dmg` from https://ollama.com/download, drag Ollama into Applications, open it once. It starts its server at login. Use either A or B, not both.

2.3 The models

```
ollama pull llama3.2:3b          # 2.0 GB, required
ollama pull nomic-embed-text     # 0.3 GB, required (must never change: the database stores 768-number vectors from it)
ollama pull qwen3:8b             # 5.2 GB, recommended second-opinion grader (section 6.2)
ollama pull qwen2.5vl:3b         # 3.2 GB, only if you will load P&ID drawings
ollama list                      # expect the models you pulled
```

Downloads go to `~/.ollama/models`. Remove one with `ollama rm <name>`; `ollama pull` brings it back.

3. Set up the project by hand (once per copy of the project)

All commands from here run in the project's `build` folder:

```
cd ~/Desktop/SHREE_INTEL_MAC_BOOK_BAKCUP_1017/DESKTOP/AI-Architect_L2_Aug_2026/project_10082026/kumar_rag-project/build
```

3.1 The settings file `.env`

Holds the database password; it stays on this Mac (it is in `.gitignore`). Skip this step if `.env` already exists.

```
PW=$(openssl rand -hex 12)
printf 'PGHOST=localhost\nPGPORT=5433\nPGUSER=ragcap\nPGPASSWORD=%s\nPGDATABASE=ragcap\n' "$PW" > .env
chmod 600 .env
cat .env | sed 's/PASSWORD=.*/PASSWORD=<hidden>/'      # expect five lines, port 5433
```

3.2 The Python environment

```
python3.12 -m venv .venv                                    # a private Python for this project
.venv/bin/pip install -r requirements.txt -r requirements-local-pg.txt    # ~2 minutes
.venv/bin/python -c "import langgraph, psycopg, pgserver, platform; print('ok', platform.machine())"   # expect: ok arm64
```

A `.venv` copied from another Mac does not work (it is tied to that Mac's Python and processor): rename it (`mv .venv .venv-old`) and run this step.

3.3 Start Ollama (if it is not running)

```
curl -s localhost:11434/api/version || (nohup ollama serve > ~/.ollama/serve.log 2>&1 &)    # expect: {"version":"0.40.x"}
```

3.4 Start the database

```
./pgctl.sh start                # first time: creates the database in .pgdata/ with the password from .env
./pgctl.sh status               # expect: pg_ctl: server is running (PID: ...)
./pgctl.sh psql -c "CREATE EXTENSION IF NOT EXISTS vector;"
./pgctl.sh psql -tAc "select extversion from pg_extension where extname='vector'"     # expect: 0.6.2
```

`pgctl.sh` uses Docker instead if the `docker` command exists; force the native database with `RAGCAP_PG=local ./pgctl.sh start`. The database listens only on 127.0.0.1 port 5433.

3.5 Create the tables and test search

```
.venv/bin/python step1_check.py
```

Expect, as the last lines: `embedding dim: 768`, `nearest: Kafka consumer lag alert threshold ...`, `STEP 1 OK`.

3.6 Load the documents and the table

```
.venv/bin/python -m ragcap.ingest data/docs_versioned        # 16 runbooks (8 topics, old and current versions)
.venv/bin/python -m ragcap.ingest data/tables                # broker_inventory.csv becomes a SQL table
./pgctl.sh psql -tAc "select (select count(*) from documents)||' docs, '|| \
  (select count(*) from chunks)||' chunks, '||(select count(*) from documents where superseded)||' superseded, '|| \
  (select count(*) from uploaded_tables)||' tables'"
```

Expect: `16 docs, 16 chunks, 8 superseded, 1 tables.`

3.7 Check the data

```
.venv/bin/python b1_check.py --no-trap --negative | tail -1   # expect: PASS
.venv/bin/python b1_check.py | tail -1                        # expect: PASS
```

3.8 Ask the first questions

```
.venv/bin/python try.py "Thanks!" "How many brokers are in rack-b?" "At what disk utilisation should a broker page?"
```

Expect three answers: a short "You're welcome" (route general), "There are 2 brokers in rack-b" (route database), and "75%" (route documents).

3.9 Confirm everything with the clean-start check

```
./services.sh check             # expect every line PASS and, at the end, CLEAN START OK
```

The setup is done.

4. Daily use

4.1 Start, check, stop

```
./services.sh start             # after a reboot or sleep: starts Ollama and the database
./services.sh check             # proves it works: ends CLEAN START OK (exit code 0)
./services.sh status            # read-only: what runs, which models are in memory
./services.sh stop              # stops the database (data is kept); add --all to stop Ollama too
```

4.2 Ask questions

How	Command	Then
Terminal	`.venv/bin/python try.py "your question"`	prints the route, each step's time, and the answer
Web page	`.venv/bin/uvicorn api:app --port 8000`	open http://127.0.0.1:8000/ ; if port 8000 is taken, use `--port 8765` and that address
Gradio chat	`.venv/bin/python ui.py`	open http://127.0.0.1:7860 ; it also lets you upload files

Stop the web page or Gradio with Ctrl+C in that Terminal window.

4.3 Add your own documents and tables

```
.venv/bin/python -m ragcap.ingest path/to/file.md          # Markdown or text
.venv/bin/python -m ragcap.ingest path/to/file.pdf         # PDF (layout-aware with Docling, section 6.3)
.venv/bin/python -m ragcap.ingest path/to/table.csv        # CSV or Excel: becomes a table the SQL agent can query
.venv/bin/python -m ragcap.ingest path/to/drawing.png      # a P&ID drawing (needs qwen2.5vl:3b)
.venv/bin/python -m ragcap.ingest path/to/folder           # everything in a folder
```

Uploads change the document count, so `./services.sh check` then shows the data line as information (`----`) instead of PASS; `./recreate.sh` puts the original 16 documents back.

4.4 Back up and restore the database

```
./pgctl.sh dump > ragcap_$(date +%Y%m%d).dump      # backup (keep it out of git; it contains document text)
./pgctl.sh restore < ragcap_20261023.dump           # restore
```

5. Measure quality (optional)

```
caffeinate -i .venv/bin/python eval.py                                  # 27 questions, under a minute
caffeinate -i .venv/bin/python eval.py --golden golden/golden_heldout.jsonl --tag unseen   # 25 unseen questions
./run_series.sh mylabel llama3.2:3b && .venv/bin/python summarize_series.py mylabel     # 3 + 2 repeated runs
```

`caffeinate -i` keeps the Mac awake; a sleeping Mac pauses a run. Results go to `results/`. Expected on this Mac: router 100%, answers 92.6% (tuned set) and 80% (unseen set), or 96% unseen with the grader from 6.2.

6. Optional extras

6.1 One-command alternative

`./recreate.sh` runs sections 3.1 to 3.8 plus the fingerprint checks and ends `RECREATE OK`. It also resets the database to the original 16 documents.

6.2 Turn on the second-opinion grader (recommended)

Add three lines to `.env`, then restart anything that is running (web page, Gradio):

```
printf 'GRADE_MODEL=qwen3:8b\nGRADE_MODE=escalate\nGRADE_MAX_TOKENS=600\n' >> .env
```

Unseen questions go from 80% to 96% correct; a hard question takes about 3 seconds instead of 1. Delete the three lines to turn it off.

6.3 Docling, for PDFs and Office files with tables

```
uv venv --python 3.12 .venv-docling
uv pip install --python .venv-docling/bin/python docling       # ~1.5 GB (includes PyTorch)
```

After that, `ragcap.ingest` uses it automatically for `.pdf`, `.docx`, `.pptx` and `.html`. The first file takes longer while Docling downloads its layout models.

7. If something goes wrong

Symptom	Fix
`command not found: ollama` or `python3.12`	open a new Terminal window, or use `~/.local/bin/ollama` and `~/.local/bin/python3.12`
`./services.sh check` shows FAIL	the line says what to do; usually `./services.sh start`
`Connection refused` on port 11434	Ollama is not running: `./services.sh start`
`Connection refused` on port 5433	the database is not running: `./pgctl.sh start`; details in `.pgdata/server.log`
`password authentication failed`	`.env` was changed after the database was created: put the old password back, or `./pgctl.sh reset` (deletes the data) and repeat 3.4 to 3.6
`address already in use` for the web page	another program uses that port: `--port 8765`
A run stopped half way	the Mac slept: run long commands with `caffeinate -i`

More cases: `docs/RUNBOOK_ragcap_operations.pdf` section 5.

8. Remove everything (if you ever need to)

```
ollama rm llama3.2:3b nomic-embed-text qwen3:8b qwen2.5vl:3b    # the models (needs Ollama running)
./services.sh stop --all                       # then stop the database and Ollama
rm -rf .venv .venv-docling .pgdata             # this project's Python and database (the database is gone after this)
rm -rf ~/.local/opt/ollama ~/.local/bin/ollama ~/.ollama       # the Ollama program and anything left in ~/.ollama
```

uv and Python 3.12 can stay; other projects can use them.